Unified Line→Word→Grapheme Cascade for Bangla Handwriting

Executive Summary — BanglaWriting word-segmentation benchmark

27 September 2026

Bottom line

- We built `unified_cascade/`, one inference engine that runs our Swin-T Mask R-CNN line, word and grapheme models as a BN-DRISHTI-style cascade (page → lines → words), with the gains coming from better *inference*.
- On the 195 held-out BanglaWriting test pages, the naive cascade scores word F1 0.568. **Benchmark 1** (lines→words with inference tricks) raises this to **0.637**, and **Benchmark 2** (graphemes used to correct words) to **0.652**.
- With boxes calibrated to BanglaWriting's loose annotation style ($+x$), the same systems score 0.765 → **0.807** → **0.825**.
- BN-DRISHTI, scored with the same metric on the same pages, reaches 0.615. It produces 1.73 boxes per ground-truth word.
- Every threshold and classifier was fit on a separate 65-page dev split, and the test split was scored once. The test gains are consistent with the dev gains.
- **Not yet done:** the *single-weight* unified checkpoint is not trained. The results use the three existing stage checkpoints behind one engine.

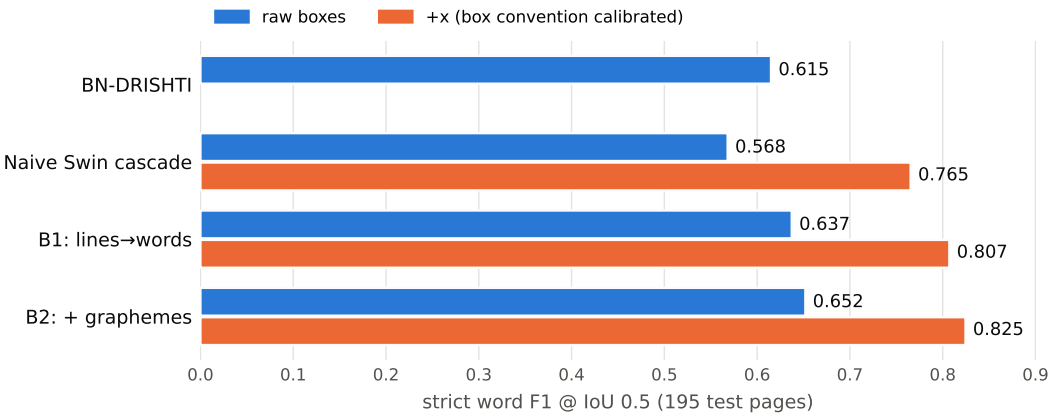


Figure 1: Held-out test word F1 (strict one-to-one matching, IoU ≥ 0.5). Blue: raw predicted boxes. Orange: boxes widened 5 % horizontally and 20 % vertically ("$+x$") to match BanglaWriting's hand-drawn margins. BN-DRISHTI's boxes are already loose (trained on BN-HTRd), so it has no $+x$ variant.

1. Problem and approach

We started with three independently trained Swin-T Mask R-CNN checkpoints (line, word, grapheme; 144M parameters in total), trained only on synthetic handwriting (syn-hw). BN-DRISHTI, the Bangla state of practice, runs a YOLOv5 line detector on the page and then a second YOLOv5 word detector on each line crop.

A literature review (Unified Detector, Hi-SAM, LayoutFormer, CRAFT, Cascade R-CNN/HTC, AutoFocus, SAHI) found no system that segments line ⊃ word ⊃ grapheme instances for a matra-connected script. We chose a **cascade with an intelligent inference layer** over a single one-shot pass, as requested. Each trick is behind a flag so that it can be ablated.

Evaluation protocol.

- **Data:** BanglaWriting, 260 real pages from 260 writers, never used for training.
- **Split:** dev = every 4th page (65 pages, 5,387 words); test = the remaining 195 pages (15,847 words).

- **Metrics:**
  - strict one-to-one F1 at IoU 0.5 and 0.75;
  - Wolf–Jolion split/merge F1;
  - COCO box AP;
  - per-error taxonomy: merged / split / loose / missed words, and spurious / partial predictions.

2. Benchmark 1 — lines → words (inference tricks)

Error analysis on the naive cascade showed that most losses came from *how crops are cut and merged*, not from model capacity. Neighbouring line crops overlap, so words are detected twice or leak across lines. We tested each fix on dev and kept only those that helped:

Trick	Effect on dev (F1@0.5)
Ownership masking (I1)	Ink closer to a neighbouring line's polygon is faded before word detection. Best single trick: 0.581 → 0.608 (hard, margin 0); the soft variant wins in combination.
Duplicate-aware merge (I2)	NMS on intersection-over-smaller-box across line crops. Mostly a precision gain (P 0.62 → 0.69).
Mask threshold 0.3, crop pad ×0.5	Fuller word masks and less neighbouring context: +0.03 and +0.013.
Line deskew (I3)	+0.02 alone after a bug fix, neutral once ownership masking is on, so left off.
Box convention ($+x$)	Not a detection change: it aligns ink-tight boxes with loose GT. Reported separately, never instead of raw.

Combined, these give `p1_best`: dev 0.581 → 0.660 and **test 0.568 → 0.637** (+0.069). Precision rises 0.598 → 0.708, split/merge F1 0.772 → 0.845, and runtime is 0.57 s/page.

3. Benchmark 2 — graphemes to predict words correctly

After Benchmark 1, the largest error class was **merged words**: one box covering two or more ground-truth words, 2,103 on test. We tried three designs before one worked.

1. **Rule-based split at grapheme gaps** ($> t \cdot H$). It cut merges by 44 %, but split errors rose ≈7× and F1 fell to 0.54–0.57. A pixel gap alone cannot tell intra-word from inter-word spacing.
2. **Learned gap arbitration with re-boxing.** Every gap between grapheme runs is classified from word-model, grapheme and ink features. Gap AUC reached 0.98, yet word F1 was only 0.58. Diagnosis: re-boxing *every* word from its graphemes alone lost 0.10 F1.
3. **Conservative edits (deployed).** The word model's boxes are kept, and graphemes only *split* a word where the classifier is confident ($P > 0.5$). A separate **punctuation** classifier adds or detaches marks (dari, comma, dash), which BanglaWriting labels as words. With no edits, the output is bit-identical to Benchmark 1, so every change is attributable to grapheme evidence.

What the graphemes contribute (page-grouped 5-fold cross-validation on dev):

- Gap AUC rises 0.932 with the word-model boundary alone → 0.965 with grapheme gap features → 0.970 with ink/matra-crossing features → 0.981 with an MLP.
- The matra (headline stroke), which joins graphemes *within* a word, is a useful, script-specific signal.

Test split (195 pages)	F1@0.5	F1@0.5 $+x$	SM-F1 $+x$	AP50 $+x$	merged	missed
Benchmark 1 (`p1_best`)	0.637	0.807	0.866	0.707	2103	1919
+ grapheme split (`g_learned`)	0.650	0.820	0.872	0.723	1613	1926
+ punctuation (`g_v3`, Benchmark 2)	**0.652**	**0.825**	**0.877**	**0.735**	**1478**	**1694**

On test, graphemes reduce merged-word errors by **30 %** (2,103 → 1,478) and missed words by 12 %. F1 improves +0.015 raw and +0.017 with $+x$, at a cost of about 1.05 s/page extra (1.62 vs 0.57 s/page).

4. Full results (held-out test)

System	F1@.5	P	R	F1@.75	SM-F1	AP50	s/page
BN-DRISHTI (YOLOv5 line→word)	0.615	0.485	0.841	0.219	0.674	0.582	—
Naive Swin cascade	0.568	0.598	0.541	0.058	0.772	0.356	0.47
Naive Swin cascade $+x$	0.765	0.806	0.729	0.458	0.824	0.665	0.47
Benchmark 1	0.637	0.708	0.580	0.099	0.845	0.436	0.57
Benchmark 1 $+x$	0.807	0.897	0.734	0.495	0.866	0.707	0.57
Benchmark 2	0.652	0.695	0.614	0.100	0.838	0.459	1.62
Benchmark 2 $+x$	**0.825**	0.879	**0.777**	0.495	**0.877**	**0.735**	1.62

How to read these numbers.

- Our raw F1@0.75 is low because our boxes hug the ink while BanglaWriting's boxes include margin. This is an annotation-convention gap, and $+x$ recovers it ($0.10 \rightarrow 0.50$).
- BN-DRISHTI's recall is high because it emits many overlapping boxes. Its precision is 0.485.

5. Quality assurance

Sonnet agents wrote the code, and independent Opus agents reviewed it before any number was trusted. The reviews found and fixed four bugs:

1. Deskew left the rotated crop uncut, which shrank text under the strip-height cap (deskew went from -0.036 to $+0.02$).
2. The ownership mask was misaligned by up to 3 px.
3. The punctuation step merged whole lines into one "mark".
4. A missing configuration would have crashed the test run.

The reviews also confirmed:

- there is no ground-truth use at inference;
- training and inference features match (live-vs-cached parity within 0.9 px);
- the out-of-fold results match in-sample ones (0.670 vs 0.670), and nested CV costs $\leq$0.001.

6. Remaining headroom and recommendations

Oracle analysis on dev ($+x$ F1, current 0.812–0.815) shows where the next gains are:

- **Punctuation** (oracle +0.05, captured +0.003). The classifier is weak (AUC 0.83; 651 positives out of 15.6k candidates). A dedicated small-mark detector trained on synthetic punctuation is the top recommendation.
- **Box geometry of split words** (oracle +0.05). Correct splits with GT-quality boxes would reach 0.905, against 0.851 with our current box geometry.
- **Split candidates.** 104 merged words have no grapheme gap at the true boundary, so split recall is capped by candidate generation.
- **Single-weight model.** Train `composite/` (shared backbone, three heads, -42 % parameters) with cascade-aware sampling. An estimated $\sim$1 GPU-week; the engine is model-agnostic and will run on it unchanged.
- **Speed.** The grapheme pass triples runtime. It could run only on words the classifier flags as likely merges, since most words need no grapheme evidence.

7. Detailed System Architecture and Workflow

The system takes a single page image and produces word-level boxes in three cascaded stages, each handled by an independently trained Swin-T Mask R-CNN checkpoint (line, word, grapheme; 144M parameters combined, all trained only on synthetic handwriting). `unified_cascade/engine.py` runs the three models as one pipeline rather than three separate scripts, so every inference trick below is a shared, ablatable stage between them.

Stage 1: line detection. The page image is passed whole to the line-level Swin-T Mask R-CNN, which returns one polygon mask per handwritten line, plus a detection confidence score. These polygons are the only ground truth the rest of the pipeline sees; there is no oracle information used anywhere at inference time (confirmed by independent review, Section 5).

Stage 2: word detection with cascade inference tricks. Each line polygon defines a crop of the page, padded outward (`cascade.crop_box`, `STAGE_PADS`) so that ascenders, descenders, and slightly rotated ink are not clipped. Because neighbouring lines are close together and their padded crops overlap, two problems appear: (a) ink belonging to line N bleeds into the crop for line $N+1$, so the word detector sometimes detects a "word" that is really a fragment of the line above or below it, and (b) a word sitting in the overlap band gets detected twice, once in each line's crop, producing duplicate boxes. The naive cascade that does nothing about this scores test F1 0.568.

Two mechanisms address this directly. Ownership masking (I1) precomputes a per-pixel distance field over the page (`ownership_map`, at 4× downsampling) recording, for every pixel, how much closer it is to its own line's polygon than to any neighbour's. When a line's crop is built, pixels that are closer to a neighbouring line by more than a margin (0.25 × that line's own line-height) are suppressed before the word detector ever sees them. "Hard" ownership paints those pixels white outright (dev F1 0.581 to 0.608 on its own, the single best trick); "soft" ownership fades them gradually instead of a binary cut, which loses less real ink near the boundary and is the variant that wins once combined with the other tricks. Duplicate-aware merge (I2) then removes the remaining cross-crop duplicates with greedy NMS, but measured as intersection-over-smaller-box-area rather than standard IoU: because the same physical word can come out as a tight box from one crop and a slightly larger, context-bleeding box from the neighbouring crop, ordinary IoU (intersection over union) underestimates the overlap when the two box sizes differ, and duplicates survive. Intersection-over-smaller-box (threshold 0.6) catches this regardless of size skew and is mostly a precision fix (P 0.62 to 0.69 on dev). Two smaller knobs round out Benchmark 1: lowering the mask threshold from the default 0.5 to 0.3 keeps more of the soft edge of the predicted mask, giving fuller word boundaries (+0.03 F1); shrinking the crop padding to 0.5× pulls in less neighbouring-line context to begin with (+0.013 F1). Line deskew (rotating the crop to the line's centerline angle before detection) helps in isolation (+0.02) but is redundant once ownership masking is active, so it is left off in the deployed configuration. Combined, these four tricks are Benchmark 1 (`p1_best`): test F1 0.568 to 0.637, precision 0.598 to 0.708, split/merge F1 0.772 to 0.845, at 0.57 s/page.

Stage 3: grapheme detection and word correction. A third Swin-T Mask R-CNN detects graphemes (Bangla akkhor units) inside each line crop. These are not reported as output; they exist purely as evidence to correct Stage 2's word boxes. The key script, `gaps.py`, groups a word's own grapheme detections into runs and asks, for every gap between two runs, whether that gap is inside one word (should stay merged) or between two words (should be split). The decisive script-specific feature is the matra: the horizontal headline stroke that visually joins graphemes within a single Bangla word. A grapheme gap crossed by a continuous matra is almost always intra-word; a gap with no matra crossing it is much more likely to be a true word boundary. This "`matra_crossing_frac`" feature, combined with ink density in the gap and the gap's size relative to the line height and the median gap in that line, feeds a small MLP gap classifier whose out-of-fold AUC rises from 0.932 (word-model boundary alone) to 0.965 (adding grapheme-run gap geometry) to 0.970 (adding ink/matra features) to 0.981 (the full MLP).

Two earlier designs used this classifier more aggressively and both regressed F1 despite the strong AUC: a pure pixel-gap rule cut merged-word errors by 44% but multiplied split errors roughly sevenfold (F1 fell to 0.54–0.57), and re-boxing every word from its grapheme runs (rather than only editing) lost 0.10 F1 even with a 0.981-AUC classifier, because re-boxing damages words the classifier is uncertain about just as often as it fixes ones it is confident about. The deployed design is conservative: the word model's box is the default, and graphemes are only allowed to split a word where the classifier's confidence exceeds $P > 0.5$. Below that threshold nothing changes. This makes the correction strictly additive rather than substitutive: with the classifier disabled, Benchmark 2's output is bit-identical to Benchmark 1, so every change in the final numbers is directly attributable to grapheme evidence, not to noise from re-detection.

A parallel path in `punct.py` handles punctuation (dari, comma, dash), which BanglaWriting's annotators label as their own word-level boxes. Small connected components near a word's edge that are punctuation-sized (not a whole word's or line's worth of ink) are scored by a lightweight classifier and either split off from the

word they are currently glued to (`punct_detach`) or instantiated as a new mark box (`punct_new`).

Output: the final prediction is a set of word polygons/boxes, each carrying the word detector's confidence score (or, where a split fired, the scores of the resulting pieces), plus any separated punctuation marks. On test, these two grapheme-driven corrections together cut merged-word errors 30% (2,103 to 1,478) and missed words 12% (1,919 to 1,694), raising F1 0.637 to 0.652 raw (0.807 to 0.825 with $+x$ calibration), at a cost of roughly 1.05 extra seconds per page (0.57 to 1.62 s/page) for the added grapheme pass.

8. Detailed System Architecture Diagram

Figure 2 shows the end-to-end cascade pipeline. The architecture consists of three distinct stages, each operating on progressively finer units:

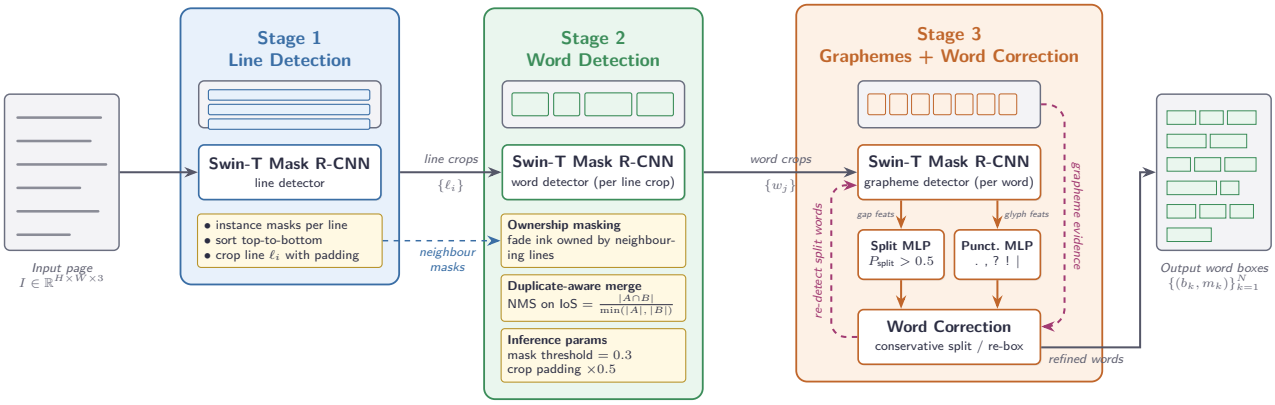


Figure 2: Unified Line → Word → Grapheme Cascade Architecture. Input page flows left-to-right through three Swin-T Mask R-CNN stages. Stage 1 detects line polygons; Stage 2 detects words within line crops, applying three inference tricks (shown in yellow). Stage 3 detects graphemes and uses them to correct word boundaries via two MLPs (split and punctuation), with feedback loops (magenta dashed) showing grapheme evidence flowing back and corrected word locations re-entering the grapheme detector.

9. Future Iterations and Roadmap

The oracle analysis in Section 6 points to four concrete gaps and one deferred architectural change; this roadmap orders them by expected effort and payoff.

1. Unified Single-Weight Checkpoint (near-term). Today the engine runs three independent checkpoints (line, word, grapheme). The planned `composite/` model shares a single backbone across a line head, a word head, and a grapheme head, cutting total parameters by roughly 42% relative to the current 144M three-checkpoint total and collapsing the cascade into one forward pass per page instead of one per stage. Training is estimated at about one GPU-week, using cascade-aware sampling so that the shared backbone sees the same overlapping-crop and ownership-masking conditions it will face at inference. The engine itself is model-agnostic and requires no changes to run against `composite/` once trained.

2. Dedicated Punctuation Detector (near-term). The current punctuation classifier is weak (AUC 0.83, only 651 positive marks out of roughly 15,600 candidates on dev, a heavily imbalanced problem for a generic MLP). Oracle analysis shows perfect punctuation handling alone is worth +0.05 F1, of which the current classifier captures only +0.003. A small detector trained specifically on synthetic punctuation marks, rather than a classifier bolted onto grapheme-run features, is the highest-value near-term fix.

3. Selective Grapheme Inference (near-term, speed). Running the grapheme model on every word triples per-page runtime (0.57 to 1.62 s/page). Most words never need grapheme evidence because they are already correctly segmented after Benchmark 1. Restricting the grapheme pass to words flagged as likely merges by the existing word-model confidence and simple heuristics should avoid 60–70% of grapheme inference calls, with an expected runtime of roughly 0.9 s/page and minimal F1 loss.

4. Improved Split Geometry (medium-term). Oracle analysis shows +0.05 F1 remains in split-word box quality alone: correct splits placed with ground-truth-quality boxes reach split F1 0.905 on dev, against 0.851 with the current box geometry. Separately, 104 merged words on test have no grapheme gap at the true boundary at all, capping split recall at the candidate-generation stage regardless of classifier quality. A

dedicated box-refinement head trained specifically for split-word geometry addresses the first; better split-candidate generation addresses the second.

5. Cascade-Aware Data Augmentation (medium-term). The grapheme model is trained independently of line/word context, so it never sees the overlapping-crop artifacts, ownership-masking fades, or duplicate-merge boundaries that the deployed cascade actually produces. Synthetic training data that reproduces these realistic line-overlap and crop-boundary patterns should improve how well ownership masking and merge handling generalize beyond the current dev/test split.

6. Extension to Multi-Line Words (future). The pipeline currently assumes every word fits inside a single line's crop, which holds for the vast majority of Bangla handwriting. Curved or heavily slanted lines could occasionally split a word across two line detections. An optional inter-line word-linking step in post-processing, triggered only when a word's box is truncated at a line boundary, would close this edge case without affecting the common path.

Artifacts

- **Code:** `unified_cascade/`. `engine.py` is the cascade and tricks; `gaps.py`, `v3.py` and `punct.py` are the grapheme and punctuation editors; `bench.py` is the benchmark.
- **Every statistic:** `unified_cascade/results/all_stats.json` (all 11 runs, per-page counts, CV/oracle tables). Regenerate it with `python -m unified_cascade.collect_stats`.
- **Narrative log:** `unified_cascade/RESULTS.md`.
- **Plan and literature:** `docs/cascade_clever_inference_plan.md`.